

Motivation

- Classical LQG assumes
 - you know the plants (A, B, C) and the noisy covariances (Q, R)
 - you get instant observations
- Bio-OFC
 - observations are delayed by d timestamps
 - (A, B, C) and noisy stats are unknown and can change
 - you still want near optimal closed-loop control, but learning must use local (pre/post) signals plus a global scalar teaching signal
- The architecture
 - A Kalman-like predictive state estimator: keeps an internal $\hat{x}_t$ and predicts current sensory input despite delays
 - A controller that maps the estimate to actions u_t , and learns its weights with a policy-gradient rule using eligibility traces

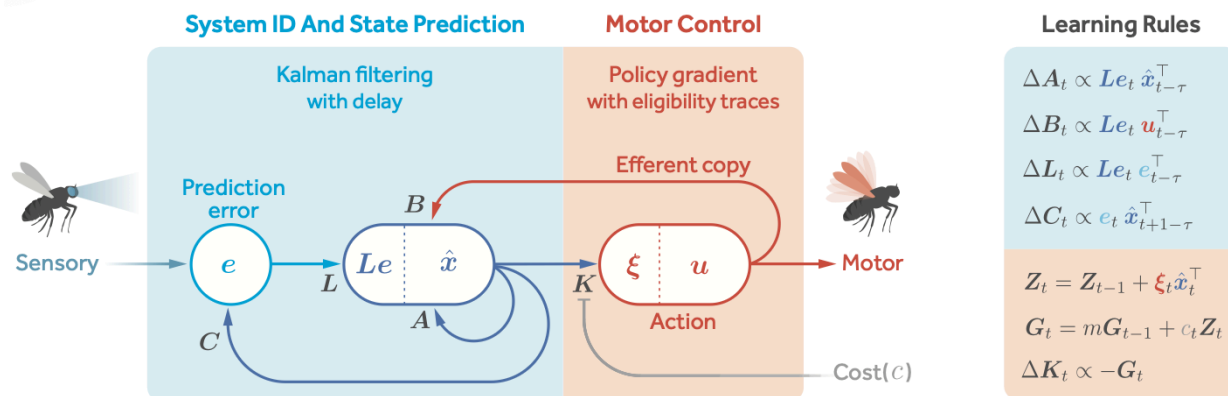


Figure 1: **The circuit and learning rules of the Bio-OFC algorithm.** Our circuit is comprised of two main parts. First (in blue), the circuit performs Kalman filtering. Then (in red), the circuit performs control using policy gradients with eligibility traces. Triangular arrowheads denote synaptic connections and the flat arrowhead denotes the modulatory effect of the cost signal.

Dynamics

- Classic Dynamics
 - $x_{t+1} = Ax_t + Bu_t + v_t$, where $v_t \sim N(0, Q)$
 - $y_t = Cx_t + w_t$, where $w_t \sim N(0, R)$
 - the sensor at time t shows d steps in the past
 - to handle delay,
 - textbook: stack the last d states to form $\bar{x}_t = [x_t^T, x_{t-1}^T, \dots, x_{t-d}^T]^T$

- Bio-OFC: keep predicting the plant forward internally until delayed y_t arrives.

- Estimation (Kalman-like) with delay $\tau = 1$

- $\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L_t(y_t - C\hat{x}_t)$ (Eq. 2)
- where $L_t = A\Sigma_t C^T (C\Sigma_t C^T + W)^{-1}$ is the KALMAN GAIN MATRIX
- where the covariance evolves as $\Sigma_{t+1} = (A - L_t C)\Sigma_t A^T + V$
- Kalman filter is optimal in the sense that it minimizes the mean squared error $E[e_t^T e_t]$ where $e_t = y_t - C\hat{x}_t$
- The policy (output feedback law) can be calculated as $u_t = -K\hat{x}_t$, where K is the CONTROL GAIN and can be solved using the Ricatti Equation.

- General delay τ

- $\hat{x}_{t+1} = A\hat{x}_t + Bu_t + L_t(y_{t+1-\tau} - C\hat{x}_{t+1-\tau})$
- where $e_t = y_{t+1-\tau} - C\hat{x}_{t+1-\tau}$ (Eq. 1)

- Stochastic Gradients

- UNDERSTAND THE DERIVATION

$$\begin{aligned}
 -\frac{\partial}{\partial A^{ij}} \frac{1}{2} \sum_k (e_t^k)^2 &= -\sum_k e_t^k \frac{\partial e_t^k}{\partial A^{ij}} = -\sum_k e_t^k \frac{\partial (y_{t+1-\tau}^k - \sum_l C^{kl} \hat{x}_{t+1-\tau}^l)}{\partial A^{ij}} = \\
 &= \sum_{k,l} e_t^k C^{kl} \frac{\partial (\sum_m A^{lm} \hat{x}_{t-\tau}^m + \sum_n B^{ln} u_t^n + \sum_p L^{lp} e_t^p)}{\partial A^{ij}} = \\
 &= \sum_{k,l,m} e_t^k C^{kl} \delta^{li} \delta^{mj} \hat{x}_{t-\tau}^m = \sum_k e_t^k C^{ki} \hat{x}_{t-\tau}^j \quad (10)
 \end{aligned}$$

○

- $-\frac{\partial}{\partial A_{ij}} \frac{1}{2} \sum_k (e_t^k)^2 = -\sum_k e_t^k \frac{\partial e_t^k}{\partial A_{ij}}$
- $= -\sum_k e_t^k \frac{\partial (y_{t+1-\tau}^k - \sum_l C^{kl} \hat{x}_{t+1-\tau}^l)}{\partial A_{ij}}$ using Eq. 1
- $= \sum_{k,l} e_t^k C^{kl} \frac{\partial (\sum_m A^{lm} \hat{x}_{t-\tau}^m + \sum_n B^{ln} u_t^n + \sum_p L^{lp} e_t^p)}{\partial A_{ij}}$ using Eq. 2 and $y_{t+1-\tau}^k$ doesn't depend on ∂A_{ij}
- $= \sum_{k,l,m} e_t^k C^{kl} \delta^{li} \delta^{mj} \hat{x}_{t-\tau}^m$

Those "deltas" are Kronecker deltas. They're just a tidy way to say "1 when the indices match, 0 otherwise." Formally:

$$\delta^{ab} = \begin{cases} 1, & a = b \\ 0, & a \neq b \end{cases}$$

They show up whenever you take the partial derivative of **one matrix element with respect to another**. Because the entries of $\hat{A}$ are independent variables,

$$\frac{\partial A^{lm}}{\partial A^{ij}} = \delta^{li} \delta^{mj}.$$

- If $(l, m) = (i, j)$ you're differentiating an entry with respect to itself $\rightarrow 1$.
- If any index differs $\rightarrow 0$.

Where it appears in your line

You're differentiating

$$\hat{x}_{t+1-\tau}^l = \sum_m A^{lm} \hat{x}_{t-\tau}^m + \dots$$

with respect to A^{ij} . Only the term $A^{lm} \hat{x}_{t-\tau}^m$ depends on A . Do it elementwise:

$$\frac{\partial}{\partial A^{ij}} (A^{lm} \hat{x}_{t-\tau}^m) = \left(\frac{\partial A^{lm}}{\partial A^{ij}} \right) \hat{x}_{t-\tau}^m = \delta^{li} \delta^{mj} \hat{x}_{t-\tau}^m.$$

Those two deltas simply **pick out** the component with $l = i$ and $m = j$.

Now plug this into the chain rule for the loss gradient:

$$\sum_{k,l,m} e_i^k C^{kl} \delta^{li} \delta^{mj} \hat{x}_{t-\tau}^m.$$

Collapse the sums using the "picker" property:

- $\sum_i C^{ki} \delta^{li} = C^{kl}$ (select column i of C),
- $\sum_m \delta^{mj} \hat{x}_{t-\tau}^m = \hat{x}_{t-\tau}^j$ (select entry j of $\hat{x}$),

to get the last line:

$$\sum_k e_i^k C^{ki} \hat{x}_{t-\tau}^j.$$

■ where

$$\circ = \sum_k e_t^k C^{ki} \hat{x}_{t-\tau}^j$$

$$-\nabla_A \frac{1}{2} \mathbf{e}_t^\top \mathbf{e}_t = \mathbf{C}^\top \mathbf{e}_t \hat{\mathbf{x}}_{t-\tau}^\top$$

$$-\nabla_B \frac{1}{2} \mathbf{e}_t^\top \mathbf{e}_t = \mathbf{C}^\top \mathbf{e}_t \mathbf{u}_{t-\tau}^\top \quad (11)$$

$$\circ -\nabla_L \frac{1}{2} \mathbf{e}_t^\top \mathbf{e}_t = \mathbf{C}^\top \mathbf{e}_t \mathbf{e}_{t-\tau}^\top$$

$$-\nabla_C \frac{1}{2} \mathbf{e}_t^\top \mathbf{e}_t = \mathbf{e}_t \hat{\mathbf{x}}_{t+1-\tau}^\top \quad (12)$$

$$\Delta \hat{\mathbf{A}}_t \propto \mathbf{L} \mathbf{e}_t \hat{\mathbf{x}}_{t-\tau}^\top \quad (13)$$

$$\Delta \hat{\mathbf{B}}_t \propto \mathbf{L} \mathbf{e}_t \mathbf{u}_{t-\tau}^\top \quad (15)$$

$$\Delta \mathbf{L}_t \propto \mathbf{L} \mathbf{e}_t \mathbf{e}_{t-\tau}^\top \quad (14)$$

$$\Delta \hat{\mathbf{C}}_t \propto \mathbf{e}_t \hat{\mathbf{x}}_{t+1-\tau}^\top \quad (16)$$

○ Understand why we can replace $\mathbf{C}^T$ with $\mathbf{L}$

■ we choose $\mathbf{M}$ so that $\mathbf{M} \mathbf{C}^T = \mathbf{L}$

- They use a standard trick: you can left-multiply the gradient by any positive-definite matrix $\mathbf{M}$ and you still decrease the loss on average

■ Concretely, they pick $\mathbf{M} = \mathbf{L} \mathbf{C}^{\dagger T}$, where $\mathbf{C}^\dagger$ is the Moore-Penrose pseudoinverse. Then, $\mathbf{M} \mathbf{C}^T = \mathbf{L} \mathbf{C}^{\dagger T} \mathbf{C}^T = \mathbf{L} \Pi$, with Π being a projection onto the column space of $\mathbf{C}^T$

■ They justify this choice explicitly: "Replacing $\mathbf{C}^T$ with $\mathbf{L}$ corresponds to multiplication of the gradients by $\mathbf{M} = \mathbf{L} \mathbf{C}^{\dagger T}$," and they initialize $\mathbf{C}$ and $\mathbf{L}$ so that $\mathbf{L} \mathbf{C}^{\dagger T}$ is positive definite (symmetric part has positive real eigenvalues). That guarantees the preconditioned step still descends the MSE.

■ Intuition in one line

- Math: precondition the gradient so you can use a local postsynaptic signal Le_t instead of the nonlocal $C^T e_t$, while still going downhill.
- Biology: synapses update by (presynaptic trace) $\times$ (postsynaptic input current), gated by a global scalar—no hidden matrix transposes required.

■